CHAPTER TWO

OBJECT-ORIENTED SYSTEM DEVELOPMENT LIFECYCLE

Diagrams in the UML

Diagram is the graphical presentation of a set of elements, most often rendered as a connected graph of vertices (things) and arcs (relationships).
In theory, a diagram may contain any combination of things and relationships.
For this reason, the UML includes **nine** such diagrams

- Class diagram
- Object diagram
- Use case diagram
- Sequence diagram
- Collaboration diagram
- State chart diagram
- Activity diagram
- Component diagram
- Deployment diagram

Class diagram

- A class diagram shows a set of classes, interfaces, and collaborations and their relationships.
- Class diagrams that include active classes address the static process view of a system.

Object diagram

- Object diagrams represent static snapshots of instances of the things found in class diagrams
- These diagrams address the static design view or static process view of a system
- An object diagram shows a set of objects and their relationships

Use case diagram

- A use case diagram shows a set of use cases and actors and their relationships
- Use case diagrams address the static use case view of a system.
- These diagrams are especially important in organizing and modeling the behaviors of a system.

Interaction Diagrams

- Both sequence diagrams and collaboration diagrams are kinds of interaction diagrams
- Interaction diagrams address the dynamic view of a system
- **A sequence diagram** is an interaction diagram that emphasizes the time-ordering of messages
- **A collaboration diagram** is an interaction diagram that emphasizes the structural organization of the objects that send and receive messages
- Sequence diagrams and collaboration diagrams are **isomorphic**, meaning that you can take one and transform it into the other.

State chart diagram

- A state chart diagram shows a state machine, consisting of states, transitions, events, and activities
- State chart diagrams address the dynamic view of a system
- They are especially important in modeling the behavior of an interface, class, or collaboration and emphasize the event-ordered behavior of an object.

Activity diagram

An activity diagram is a special kind of a state chart diagram that shows the flow from activity to activity within a system

Activity diagrams address the dynamic view of a system. They are especially important in modeling the function of a system and emphasize the flow of control among objects.

Component diagram

- A component diagram shows the organizations and dependencies among a set of components.
- Component diagrams address the static implementation view of a system
- They are related to class diagrams in that a component typically maps to one or more classes, interfaces, or collaborations.

Deployment diagram

- A deployment diagram shows the configuration of run-time processing nodes and the components that live on them
- Deployment diagrams address the static deployment view of an architecture

Rules of the UML

The UML has semantic rules for

1. Names What you can call things, relationships, and diagrams
2. Scope The context that gives specific meaning to a name
3. Visibility How those names can be seen and used by others
4. Integrity How things properly and consistently relate to one another
5. Execution What it means to run or simulate a dynamic model

Models built during the development of a software-intensive system tend to evolve and may be viewed by many stakeholders in different ways and at different times. For this reason, it is common for the development team to not only build models that are well-formed, but also to build models that are

1. Elided Certain elements are hidden to simplify the view
2. Incomplete Certain elements may be missing
3. Inconsistent The integrity of the model is not guaranteed

Common Mechanisms in the UML

UML is made simpler by the presence of **four** common mechanisms that apply consistently throughout the language.

1. Specifications
2. Adornments
3. Common divisions
4. Extensibility mechanisms

Specification

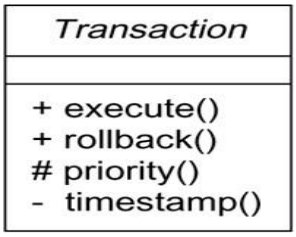
- They provides a textual statement of the syntax and semantics of that building block.

- The UML's specifications provide a semantic backplane that contains all the parts of all the models of a system, each part related to one another in a consistent fashion

Adornments

- Most elements in the UML have a unique and direct graphical notation that provides a visual representation of the most important aspects of the element.
- A class's specification may include other details, such as whether it is abstract or the visibility of its attributes and operations.

- Many of these details can be rendered as graphical or textual adornments to the class's basic rectangular notation.

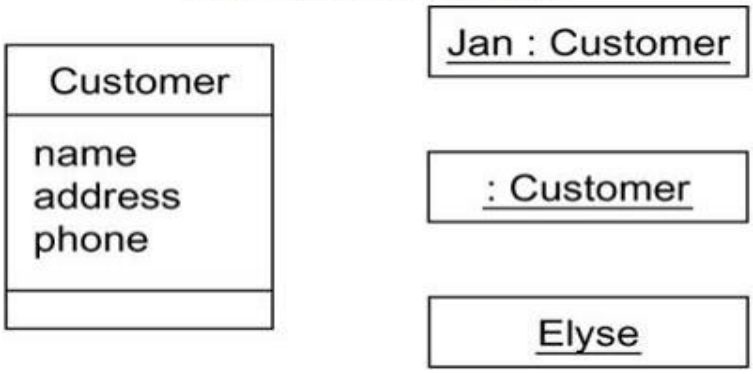


Common Divisions

In modeling object-oriented systems, the world often gets divided in at least a couple of ways.

First, there is the division of class and object. A class is an abstraction; an object is one concrete manifestation of that abstraction. In the UML, you can model classes as well as objects, as shown in Figure 2-17

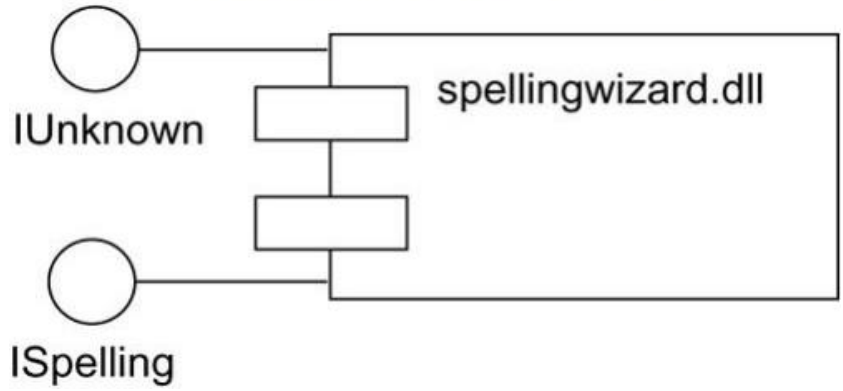
Figure 2-17 Classes And Objects



In this figure, there is one class, named Customer, together with three objects: Jan (which is marked explicitly as being a Customer object), :Customer (an anonymous Customer object), and Elyse (which in its specification is marked as being a kind of Customer object, although it's not shown explicitly here).

Second, there is the separation of interface and implementation. An interface declares a contract, and an implementation represents one concrete realization of that contract, responsible for faithfully carrying out the interface's complete semantics. In the UML, you can model both interfaces and their implementations, as shown in Figure 2-18.

Figure 2-18 Interfaces And Implementations



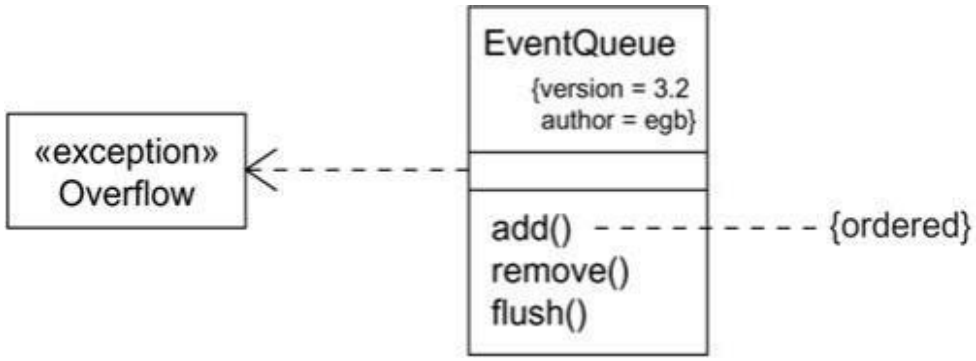
In this figure, there is one component named spellingwizard.dll that implements two interfaces, IUnknown and ISpelling.

Extensibility Mechanisms

The UML's extensibility mechanisms include

1. Stereotypes
2. Tagged values
3. Constraints

- **Stereotype** extends the vocabulary of the UML, allowing you to create new kinds of building blocks that are derived from existing ones but that are specific to your problem
- **A tagged value** extends the properties of a UML building block, allowing you to create new information in that element's specification
- **A constraint** extends the semantics of a UML building block, allowing you to add new rules or modify existing ones. Examples for these are



Architecture

A system's architecture is perhaps the most important artifact that can be used to manage these different viewpoints and so control the iterative and incremental development of a system throughout its life cycle.

Architecture is the set of significant decisions about

The organization of a software system

The selection of the structural elements and their interfaces by which the system is composed

Their behavior, as specified in the collaborations among those elements

The composition of these structural and behavioral elements into progressively larger subsystems

The architectural style that guides this organization: the static and dynamic elements and their interfaces, their collaborations, and their composition.

Software architecture is not only concerned with structure and behavior, but also with usage, functionality, performance, resilience, reuse, comprehensibility, economic and technology constraints and trade-offs, and aesthetic concerns.

Figure 2-20 Modeling a System's Architecture

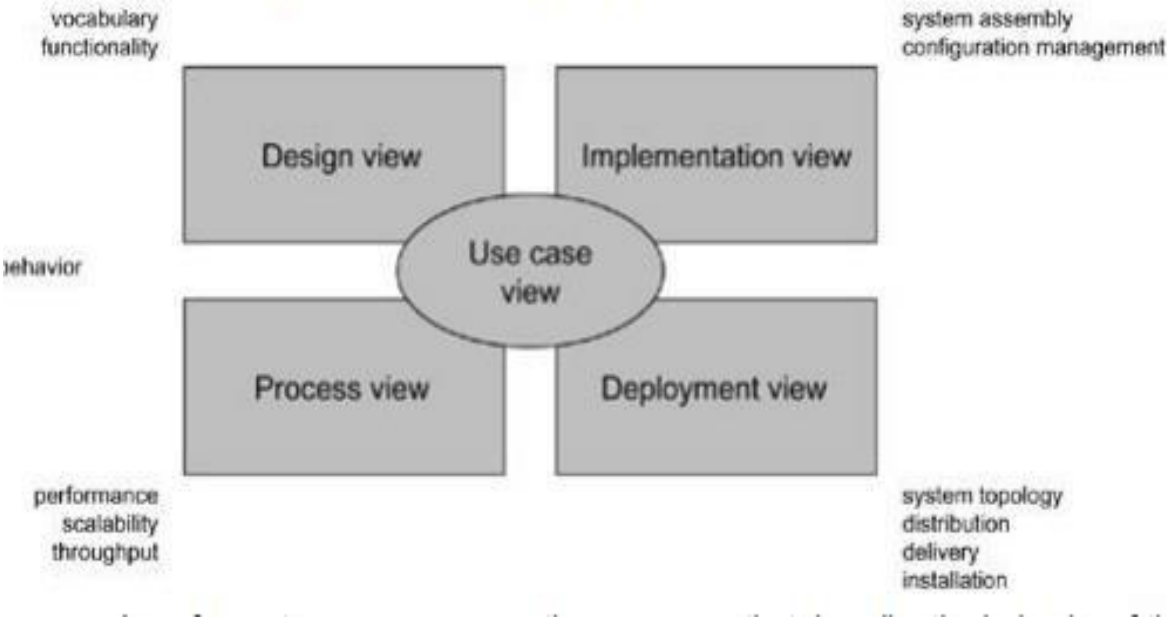


Fig: Modeling a System's Architecture

Use case view

- The use case view of a system encompasses the use cases that describe the behavior of the system as seen by its end users, analysts, and testers.
  - With the UML, the static aspects of this view are captured in use case diagrams
- The dynamic aspects of this view are captured in interaction diagrams, state chart diagrams, and activity diagrams.

Design View

- The design view of a system encompasses the classes, interfaces, and collaborations that form the vocabulary of the problem and its solution.
- This view primarily supports the functional requirements of the system, meaning the services that the system should provide to its end users.

Process View

- The process view of a system encompasses the threads and processes that form the system's concurrency and synchronization mechanisms.
- This view primarily addresses the performance, scalability, and throughput of the system.

Implementation View

- The implementation view of a system encompasses the components and files that are used to assemble and release the physical system.
- This view primarily addresses the configuration management of the system's releases, made up of somewhat independent components and files that can be assembled in various ways to produce a running system.

Deployment View

- The deployment view of a system encompasses the nodes that form the system's hardware topology on which the system executes.
- This view primarily addresses the distribution, delivery, and installation of the parts that make up the physical system.
- With the UML, the static aspects of this view are captured in deployment diagrams; the dynamic aspects of this view are captured in interaction diagrams, state chart diagrams, and activity diagrams.

- Each of these five views can stand alone so that different stakeholders can focus on the issues of the system's architecture that most concern them.

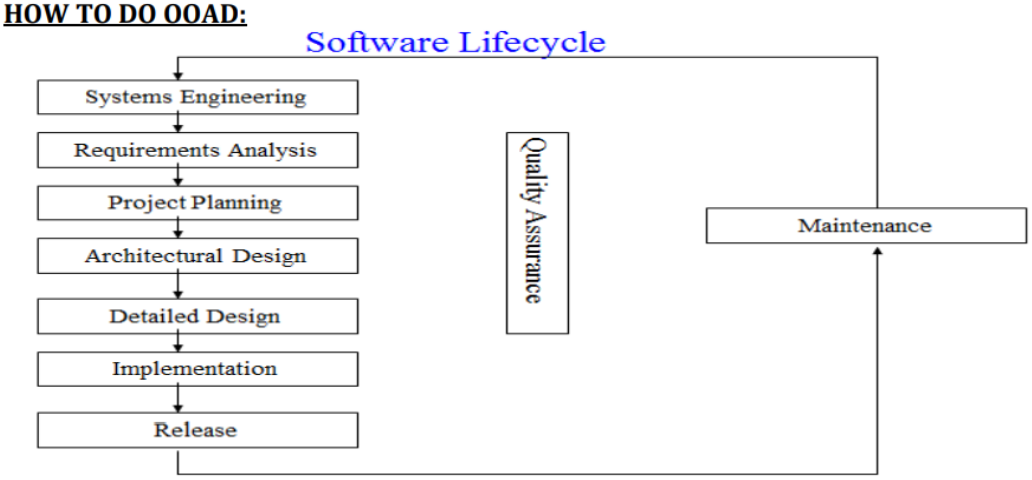
- These five views also interact with one another• nodes in the deployment view hold components in the implementation view that, in turn, represent the physical realization of classes, interfaces, collaborations, and active classes from the design and process views.

- The UML permits you to express every one of these five views and their interactions.

Software Development Life Cycle

- The UML is largely process-independent, meaning that it is not tied to any particular software development life cycle.
- However, to get the most benefit from the UML, you should consider a process that is
    - Use case driven
    - Architecture-centric
    - Iterative and incremental

HOW TO DO OOAD:



Use case driven

- It means that use cases are used as a primary artifact for establishing the desired behavior of the system, for verifying and validating the system's architecture, for testing, and for communicating among the stakeholders of the project.

Architecture-centric

- It means that a system's architecture is used as a primary artifact for Conceptualizing, constructing, managing, and evolving the system under development.

An iterative process

- It is one that involves managing a stream of executable releases.

An incremental process

- It is one that involves the continuous integration of the system's architecture to produce these releases, with each new release embodying incremental improvements over the other.

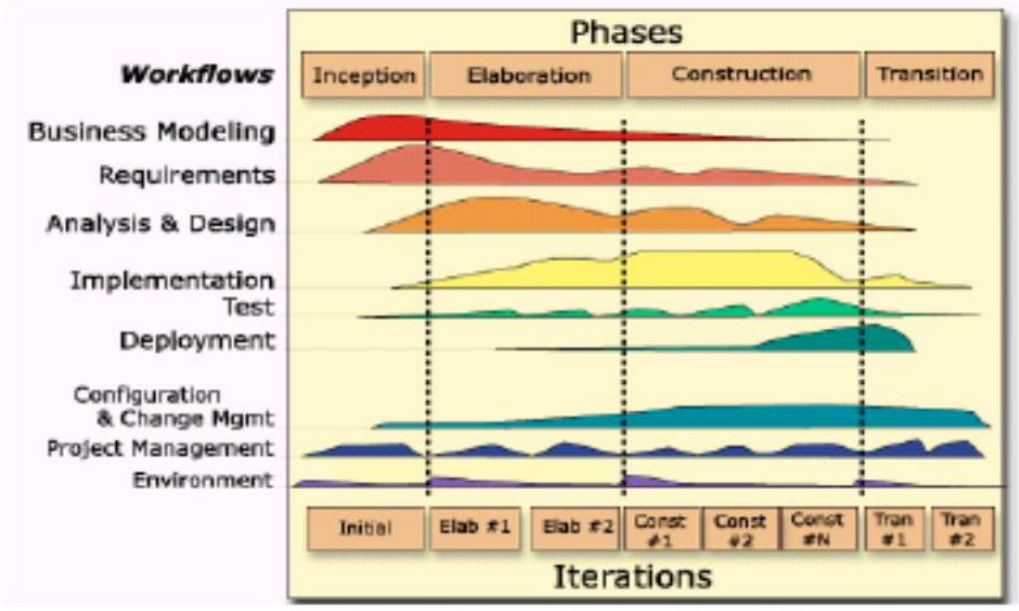
- Together, an iterative and incremental process is *risk-driven,* meaning that each new release is focused on attacking and reducing the most significant risks to the success of the project. This use case driven, architecture-centric, and iterative/incremental process can be broken into phases.

- **A *phase*** is the span of time between two major milestones of the process, when a well-defined set of objectives are met, artifacts are completed, and decisions are made whether to move into the next phase.

- As below figure shows, there are **four** phases in the software development life cycle: inception, elaboration, construction, and transition. In the diagram, workflows are plotted against these phases, showing their varying degrees of focus over time.

-

WORKFLOWS IN OOAD:



RUP Overview Diagram

Fig: Figure Software Development Life Cycle Inception phase:

- It is the first phase of the process
- When the seed idea for the development is brought up to the point of being• at least internally• sufficiently well-founded to warrant entering into the elaboration phase.

Elaboration phase:

- It is the second phase of the process
- When the product vision and its architecture are defined.
- In this phase, the system's requirements are articulated, prioritized, and baselined.
- A system's requirements may range from general vision statements to precise evaluation criteria, each specifying particular functional or nonfunctional behavior and each providing a basis for testing.

Construction phase:

- It **i**s the third phase of the process.
- When the software is brought from an executable architectural baseline to being ready to be transitioned to the user community.
- The system's requirements and especially its evaluation criteria are constantly reexamined against the business needs of the project, and resources are allocated as appropriate to actively attack risks to the project.

Transition phase:

- It is the fourth phase of the process.
- When the software is turned into the hands of the user community.
- The software development process end here, for even during this phase, the system is continuously improved, bugs are eradicated, and features that didn't make an earlier release are added.

- One element that distinguishes this process and that cuts across all four phases is an iteration.
- An iteration is a distinct set of activities, with a baselined plan and evaluation criteria that result in a release, either internal or external.
- This means that the software development life cycle can be characterized as involving a continuous stream of executable releases of the system's architecture.
- It is this emphasis on architecture as an important artifact that drives the UML to focus on modeling the different views of a system's architecture.